In [1]:
```
!pip install pandas numpy scikit-learn xgboost fastapi uvicorn nest-asyncio matp
```

```
Requirement already satisfied: pandas in c:\users\monisha v\anaconda3\lib\site-pa
ckages (2.3.3)
Requirement already satisfied: numpy in c:\users\monisha v\anaconda3\lib\site-pac
kages (2.3.5)
Requirement already satisfied: scikit-learn in c:\users\monisha v\anaconda3\lib\s
ite-packages (1.7.2)
Requirement already satisfied: xgboost in c:\users\monisha v\anaconda3\lib\site-p
ackages (3.2.0)
Requirement already satisfied: fastapi in c:\users\monisha v\anaconda3\lib\site-p
ackages (0.135.3)
Requirement already satisfied: uvicorn in c:\users\monisha v\anaconda3\lib\site-p
ackages (0.44.0)
Requirement already satisfied: nest-asyncio in c:\users\monisha v\anaconda3\lib\s
ite-packages (1.6.0)
Requirement already satisfied: matplotlib in c:\users\monisha v\anaconda3\lib\sit
e-packages (3.10.6)
Requirement already satisfied: seaborn in c:\users\monisha v\anaconda3\lib\site-p
ackages (0.13.2)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\monisha v\anaco
nda3\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\monisha v\anaconda3\lib\s
ite-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\monisha v\anaconda3\lib
\site-packages (from pandas) (2025.2)
Requirement already satisfied: scipy>=1.8.0 in c:\users\monisha v\anaconda3\lib\s
ite-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in c:\users\monisha v\anaconda3\lib
\site-packages (from scikit-learn) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\monisha v\anacond
a3\lib\site-packages (from scikit-learn) (3.5.0)
Requirement already satisfied: starlette>=0.46.0 in c:\users\monisha v\anaconda3
\lib\site-packages (from fastapi) (1.0.0)
Requirement already satisfied: pydantic>=2.9.0 in c:\users\monisha v\anaconda3\li
b\site-packages (from fastapi) (2.12.4)
Requirement already satisfied: typing-extensions>=4.8.0 in c:\users\monisha v\ana
conda3\lib\site-packages (from fastapi) (4.15.0)
Requirement already satisfied: typing-inspection>=0.4.2 in c:\users\monisha v\ana
conda3\lib\site-packages (from fastapi) (0.4.2)
Requirement already satisfied: annotated-doc>=0.0.2 in c:\users\monisha v\anacond
a3\lib\site-packages (from fastapi) (0.0.4)
Requirement already satisfied: click>=7.0 in c:\users\monisha v\anaconda3\lib\sit
e-packages (from uvicorn) (8.2.1)
Requirement already satisfied: h11>=0.8 in c:\users\monisha v\anaconda3\lib\site-
packages (from uvicorn) (0.16.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\monisha v\anaconda3\l
ib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\monisha v\anaconda3\lib\s
ite-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\monisha v\anaconda3
\lib\site-packages (from matplotlib) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\monisha v\anaconda3
\lib\site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\monisha v\anaconda3\li
b\site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\monisha v\anaconda3\lib\site
-packages (from matplotlib) (12.0.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\monisha v\anaconda3\l
ib\site-packages (from matplotlib) (3.2.5)
Requirement already satisfied: colorama in c:\users\monisha v\anaconda3\lib\site-
packages (from click>=7.0->uvicorn) (0.4.6)
```

```
Requirement already satisfied: annotated-types>=0.6.0 in c:\users\monisha v\anaco
nda3\lib\site-packages (from pydantic>=2.9.0->fastapi) (0.6.0)
Requirement already satisfied: pydantic-core==2.41.5 in c:\users\monisha v\anacon
da3\lib\site-packages (from pydantic>=2.9.0->fastapi) (2.41.5)
Requirement already satisfied: six>=1.5 in c:\users\monisha v\anaconda3\lib\site-
packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: anyio<5,>=3.6.2 in c:\users\monisha v\anaconda3\li
b\site-packages (from starlette>=0.46.0->fastapi) (4.10.0)
Requirement already satisfied: idna>=2.8 in c:\users\monisha v\anaconda3\lib\site
-packages (from anyio<5,>=3.6.2->starlette>=0.46.0->fastapi) (3.11)
Requirement already satisfied: sniffio>=1.1 in c:\users\monisha v\anaconda3\lib\s
ite-packages (from anyio<5,>=3.6.2->starlette>=0.46.0->fastapi) (1.3.0)
```

In [2]:

```python
# Data Manipulation
import pandas as pd
import numpy as np

# Machine Learning
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error

# API & Backend
from fastapi import FastAPI
import uvicorn
import nest_asyncio

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Set visual style for later graphs
sns.set_theme(style="darkgrid")

print("✅ Phase 1 Complete: All libraries successfully imported!")
print(f"Pandas Version: {pd.__version__}")
print(f"XGBoost Version: {xgb.__version__}")
```

```
✅ Phase 1 Complete: All libraries successfully imported!
Pandas Version: 2.3.3
XGBoost Version: 3.2.0
```

In [21]:

```python
import os

print("Building data folder and writing CSV files...")

# 1. Create the 'data' folder (if it doesn't already exist)
os.makedirs('data', exist_ok=True)

# 2. Write the Market Data CSV
market_data = """Sl no.,District Name,Market Name,Commodity,Variety,Grade,Min Pr
1,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1100,1300,1200,01-Mar-2026
2,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1100,1350,1200,02-Mar-2026
3,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1050,1300,1150,03-Mar-2026
4,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1000,1200,1100,04-Mar-2026
5,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,950,1150,1050,05-Mar-2026
6,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,900,1100,1000,06-Mar-2026
7,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,900,1100,1000,07-Mar-2026
8,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,850,1050,950,08-Mar-2026
9,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,850,1050,950,09-Mar-2026
```

```
10,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,900,1100,1000,10-Mar-2026
11,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,950,1200,1050,11-Mar-2026
12,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,950,1250,1100,12-Mar-2026
13,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1000,1300,1150,13-Mar-2026
14,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1000,1350,1200,14-Mar-2026
15,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1050,1400,1200,15-Mar-2026
16,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1050,1400,1250,16-Mar-2026
17,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1100,1500,1300,17-Mar-2026
18,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1150,1500,1300,18-Mar-2026
19,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1150,1550,1350,19-Mar-2026
20,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1200,1600,1400,20-Mar-2026
21,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1250,1650,1450,22-Mar-2026
22,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1250,1650,1450,23-Mar-2026
23,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1200,1550,1400,24-Mar-2026
24,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1150,1500,1350,25-Mar-2026
25,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1100,1450,1300,26-Mar-2026
26,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1050,1400,1250,28-Mar-2026
27,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1050,1350,1200,29-Mar-2026
28,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1100,1400,1250,30-Mar-2026
29,Krishnagiri,Hosur,Tomato,Hybrid,FAQ,1150,1450,1300,31-Mar-2026"""

with open('data/agmarknet_march.csv', 'w') as f:
    f.write(market_data)

# 3. Write the Weather Data CSV
weather_data = """YEAR,MO,DY,T2M_MAX,RH2M,ALLSKY_SFC_SW_DWN
2026,3,1,34.5,45.2,22.1
2026,3,2,35.1,44.8,22.3
2026,3,3,34.8,46.1,21.8
2026,3,4,35.5,43.5,22.5
2026,3,5,36.0,42.0,22.8
2026,3,6,36.2,41.5,23.0
2026,3,7,35.9,42.2,22.6
2026,3,8,36.5,40.1,23.2
2026,3,9,36.8,39.5,23.5
2026,3,10,37.1,38.2,23.6
2026,3,11,37.0,38.8,23.1
2026,3,12,37.5,37.5,23.8
2026,3,13,38.0,36.1,24.0
2026,3,14,38.2,35.5,24.1
2026,3,15,38.5,34.8,24.3
2026,3,16,37.8,36.0,23.5
2026,3,17,38.1,35.2,24.0
2026,3,18,38.6,34.1,24.5
2026,3,19,39.0,33.5,24.8
2026,3,20,39.2,33.0,25.0
2026,3,21,39.5,32.1,25.2
2026,3,22,39.8,31.5,25.4
2026,3,23,40.1,30.8,25.6
2026,3,24,40.5,30.2,25.8
2026,3,25,40.8,29.5,26.0
2026,3,26,41.0,29.0,26.2
2026,3,27,41.2,28.5,26.3
2026,3,28,40.9,29.1,26.1
2026,3,29,40.5,30.0,25.8
2026,3,30,41.1,28.8,26.2
2026,3,31,41.5,28.0,26.5"""

with open('data/nasa_power_weather.csv', 'w') as f:
    f.write(weather_data)
```

```
print("✅ Success! The 'data' folder and both CSV files are now in the correct l
```

```
Building data folder and writing CSV files...
✅ Success! The 'data' folder and both CSV files are now in the correct locatio
n.
```

In [2]:
```
import os
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

print("🚜 Generating 2 Years of Enterprise Agricultural Data...")

# 1. Setup 2 Years of Dates (Jan 2024 to Dec 2025)
dates = pd.date_range(start="2024-01-01", end="2025-12-31")
days = len(dates)
os.makedirs('data', exist_ok=True)

# 2. Simulate Realistic Climate Data (Sine Waves for Seasons)
# Summer peaks around day 150 (May). Base temp 32, amplitude 5.
temps = 32 + 5 * np.sin(2 * np.pi * (dates.dayofyear - 60) / 365) + np.random.nc
temps = np.clip(temps, 25, 42.5) # Keep within realistic bounds

# Humidity drops when temp rises
humidity = 80 - (temps - 25) * 2.5 + np.random.normal(0, 5, days)
humidity = np.clip(humidity, 20, 95)

weather_df = pd.DataFrame({
    'YEAR': dates.year,
    'MO': dates.month,
    'DY': dates.day,
    'T2M_MAX': np.round(temps, 1),
    'RH2M': np.round(humidity, 1),
    'ALLSKY_SFC_SW_DWN': np.round(np.random.normal(23, 2, days), 1)
})
weather_df.to_csv('data/nasa_power_weather.csv', index=False)

# 3. Simulate Realistic Market Economics
# Base price is 1000. Prices rise naturally in late summer.
base_price = 1000
seasonal_price = 200 * np.sin(2 * np.pi * (dates.dayofyear - 120) / 365)

# Causal Rule: If temp goes over 38°C (Extreme Heat), crops die, supply drops, p
heat_stress_spike = np.where(temps > 38, (temps - 38) * 150, 0)

prices = base_price + seasonal_price + heat_stress_spike + np.random.normal(0, 8
prices = np.clip(prices, 500, 3500)

market_df = pd.DataFrame({
    'Sl no.': range(1, days + 1),
    'District Name': 'Krishnagiri',
    'Market Name': 'Hosur',
    'Commodity': 'Tomato',
    'Variety': 'Hybrid',
    'Grade': 'FAQ',
    'Min Price (Rs./Quintal)': np.round(prices * 0.8),
    'Max Price (Rs./Quintal)': np.round(prices * 1.2),
    'Modal Price (Rs./Quintal)': np.round(prices),
    'Price Date': dates.strftime('%d-%b-%Y')
```

```
})
# We keep the old filename so your main code doesn't have to change!
market_df.to_csv('data/agmarknet_march.csv', index=False)

print(f"✅ Successfully generated {days} days of data and saved to CSVs!")
```

```
🚜 Generating 2 Years of Enterprise Agricultural Data...
✅ Successfully generated 731 days of data and saved to CSVs!
```

In [ ]:

```
# ==========================================
# AGRO-PRICE GUARD: BACKEND (FastAPI + AI)
# ==========================================

import pandas as pd
import numpy as np
import joblib
import nest_asyncio
import requests
from xgboost import XGBRegressor
from sklearn.metrics import r2_score, mean_absolute_error # <-- NEW metrics
from fastapi import FastAPI
from pydantic import BaseModel, Field
from uvicorn import Config, Server

print("🚀 INITIALIZING AGRO-PRICE GUARD BACKEND...")

# ------------------------------------------------------------
# PHASE 1 & 2: LOAD THE 2-YEAR DATASET
# ------------------------------------------------------------
df_market = pd.read_csv('data/agmarknet_march.csv')
df_market['Date'] = pd.to_datetime(df_market['Price Date'])
df_market = df_market.drop(columns=['Price Date'])

df_weather = pd.read_csv('data/nasa_power_weather.csv')
df_weather['Date'] = pd.to_datetime(df_weather[['YEAR', 'MO', 'DY']].astype(str)
df_weather = df_weather.drop(columns=['YEAR', 'MO', 'DY'])

# ------------------------------------------------------------
# PHASE 3: THE GRAND MERGE
# ------------------------------------------------------------
df_master = pd.merge(df_market, df_weather, on='Date', how='right')
df_master = df_master.sort_values('Date').reset_index(drop=True)
df_master.set_index('Date', inplace=True)
df_master['Modal Price (Rs./Quintal)'] = df_master['Modal Price (Rs./Quintal)'].
df_master['Market Name'] = df_master['Market Name'].ffill()
df_master['Commodity'] = df_master['Commodity'].ffill()
df_master['District Name'] = df_master['District Name'].ffill()

# ------------------------------------------------------------
# PHASE 4: FEATURE ENGINEERING & MODEL TRAINING
# ------------------------------------------------------------
df_ml = df_master.copy()

# Heat Stress Indicators
df_ml['is_extreme_heat'] = (df_ml['T2M_MAX'] >= 38).astype(int)
df_ml['heat_stress_3d'] = df_ml['is_extreme_heat'].rolling(window=3).sum()

# Lag Features
for i in range(1, 4):
    df_ml[f'price_lag_{i}'] = df_ml['Modal Price (Rs./Quintal)'].shift(i)
```

```
    df_ml[f'temp_lag_{i}'] = df_ml['T2M_MAX'].shift(i)
    df_ml[f'humidity_lag_{i}'] = df_ml['RH2M'].shift(i)

df_ml.dropna(inplace=True)

target_col = 'Modal Price (Rs./Quintal)'
columns_to_hide = [
    target_col, 'Sl no.', 'District Name', 'Market Name',
    'Commodity', 'Variety', 'Grade',
    'Min Price (Rs./Quintal)', 'Max Price (Rs./Quintal)',
    'ALLSKY_SFC_SW_DWN'
]

features = [col for col in df_ml.columns if col not in columns_to_hide]
X = df_ml[features]
y = df_ml[target_col]

split_index = int(len(df_ml) * 0.75)
X_train, X_test = X.iloc[:split_index], X.iloc[split_index:]
y_train, y_test = y.iloc[:split_index], y.iloc[split_index:]

# Train & Save
model = XGBRegressor(n_estimators=100, learning_rate=0.1, max_depth=3, random_st
model.fit(X_train, y_train)

# --- PROVE THE AI IS FIXED ---
predictions = model.predict(X_test)
r2 = r2_score(y_test, predictions)
mae = mean_absolute_error(y_test, predictions)

joblib.dump(model, 'krishnagiri_tomato_xgboost.pkl')
print(f"✅ AI Brain Trained & Saved!")
print(f"📊 New R-Squared Score: {round(r2 * 100, 2)}%")
print(f"📊 Mean Absolute Error: ₹{round(mae, 2)} per Quintal")

# ----------------------------------------------------------
# PHASE 5: THE FASTAPI SERVICE & LOGISTICS ENGINE
# ----------------------------------------------------------
app = FastAPI(title="Agro-Price Guard", description="Krishnagiri Market Mitigati
loaded_model = joblib.load('krishnagiri_tomato_xgboost.pkl')

class MarketDataInput(BaseModel):
    current_temp: float = Field(..., description="Today's Max Temperature in Cel
    current_humidity: float = Field(..., description="Today's Relative Humidity
    past_temps: list[float] = Field(..., description="Max temps for the last 3 d
    past_humidities: list[float] = Field(..., description="Humidity for the last
    past_prices: list[float] = Field(..., description="Mandi prices for the last

class LiveMarketInput(BaseModel):
    past_prices: list[float] = Field(..., description="Mandi prices for the last

def get_strategy(predicted_price):
    base_price = 1200
    drop_pct = ((base_price - predicted_price) / base_price) * 100

    if drop_pct >= 15:
        return {
            "Alert Level": "🔴 CRITICAL",
            "Action": "DIVERT TO PROCESSING",
            "Details": "Massive market glut predicted. Bypass Hosur Mandi and ro
```

```
        }
    elif 5 <= drop_pct < 15:
        return {
            "Alert Level": "🟡 WARNING",
            "Action": "COLD STORAGE",
            "Details": "Moderate price drop predicted. Store harvest at Kaveripa
        }
    else:
        return {
            "Alert Level": "🟢 NORMAL",
            "Action": "MANDI SALE",
            "Details": "Market stable. Proceed with standard auction at Hosur Uz
        }

@app.post("/predict-and-route", tags=["Logistics Engine"])
def predict_route(data: MarketDataInput):
    all_temps = [data.current_temp] + data.past_temps
    is_extreme = [1 if t >= 38 else 0 for t in all_temps]
    heat_stress_3d = sum(is_extreme[:3])

    feature_dict = {
        'T2M_MAX': [data.current_temp], 'RH2M': [data.current_humidity],
        'is_extreme_heat': [1 if data.current_temp >= 38 else 0], 'heat_stress_3
        'price_lag_1': [data.past_prices[0]], 'temp_lag_1': [data.past_temps[0]]
        'price_lag_2': [data.past_prices[1]], 'temp_lag_2': [data.past_temps[1]]
        'price_lag_3': [data.past_prices[2]], 'temp_lag_3': [data.past_temps[2]]
    }

    input_df = pd.DataFrame(feature_dict)[features]
    predicted_val = loaded_model.predict(input_df)[0]
    return {"Status": "Success", "AI_Predicted_Price_INR": round(float(predicted

@app.post("/predict-live", tags=["Live Integration"])
def predict_live(data: LiveMarketInput):
    weather_url = "https://api.open-meteo.com/v1/forecast?latitude=12.5186&longi
    res = requests.get(weather_url).json()

    temps = res['daily']['temperature_2m_max']
    hums = res['hourly']['relative_humidity_2m']
    daily_hums = [hums[12], hums[36], hums[60], hums[84]]

    current_temp = temps[3]
    current_humidity = daily_hums[3]
    past_temps = [temps[2], temps[1], temps[0]]
    past_humidities = [daily_hums[2], daily_hums[1], daily_hums[0]]

    all_temps = [current_temp] + past_temps
    is_extreme = [1 if t >= 38 else 0 for t in all_temps]
    heat_stress_3d = sum(is_extreme[:3])

    feature_dict = {
        'T2M_MAX': [current_temp], 'RH2M': [current_humidity],
        'is_extreme_heat': [1 if current_temp >= 38 else 0], 'heat_stress_3d': [
        'price_lag_1': [data.past_prices[0]], 'temp_lag_1': [past_temps[0]], 'hu
        'price_lag_2': [data.past_prices[1]], 'temp_lag_2': [past_temps[1]], 'hu
        'price_lag_3': [data.past_prices[2]], 'temp_lag_3': [past_temps[2]], 'hu
    }

    input_df = pd.DataFrame(feature_dict)[features]
    predicted_val = loaded_model.predict(input_df)[0]
```

```
    return {
        "Status": "Success",
        "Fetched_Weather": {"Location": "Krishnagiri", "Today_Max_Temp": current
        "AI_Predicted_Price_INR": round(float(predicted_val), 2),
        "Recommendation": get_strategy(predicted_val)
    }

# Launch Server (Jupyter Safe)
nest_asyncio.apply()
print("🌐 BACKEND IS LIVE! Ready to receive data.")

config = Config(app=app, host="127.0.0.1", port=8000, log_level="info")
server = Server(config)
await server.serve()
```

```
🚀 INITIALIZING AGRO-PRICE GUARD BACKEND...
✅ AI Brain Trained & Saved!
📊 New R-Squared Score: 57.23%
📊 Mean Absolute Error: ₹75.0 per Quintal
INFO:     Started server process [8228]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
🌐 BACKEND IS LIVE! Ready to receive data.
INFO:     127.0.0.1:54645 - "POST /predict-live HTTP/1.1" 200 OK
INFO:     127.0.0.1:57287 - "POST /predict-live HTTP/1.1" 200 OK
INFO:     127.0.0.1:65326 - "POST /predict-live HTTP/1.1" 200 OK
INFO:     127.0.0.1:62622 - "POST /predict-live HTTP/1.1" 200 OK
INFO:     127.0.0.1:53569 - "POST /predict-live HTTP/1.1" 200 OK
INFO:     127.0.0.1:64422 - "POST /predict-live HTTP/1.1" 200 OK
```